

Week 4 Technical Report - Graph Algorithms

Project repository (complete source, tests, benchmarks and results):

<https://github.com/RDeibel2025/CTX-Advanced-Algorithms>

Graphs: `src/graphs/` · demonstration: `examples/week4_demo.py`

Benchmark: `benchmarks/week4_graph_benchmark.py`

Robert Deibel · CSC 5300 Advanced Algorithms · Concordia University Texas · Fall 2026



Complete project repository

<https://github.com/RDeibel2025/CTX-Advanced-Algorithms>

1. Executive Summary

BFS, iterative DFS and Dijkstra are one loop with three different containers, and the container decided the bill as well as the order: trading Dijkstra's binary heap for a linearly scanned list left every answer identical and made it 26.21x slower at $V = 2,000$. The adjacency matrix cost 35.6x the memory at $V = 10,000$ and bought no lookup advantage.

2. Methodology

Environment. Apple M2 Max, 12 cores, 64 GB RAM; macOS 14.5 (arm64); CPython 3.12.4. Timing runs through the Week 1 `AlgorithmBenchmark: time.perf_counter()`, warm-ups discarded, mean, SD, min and max reported.

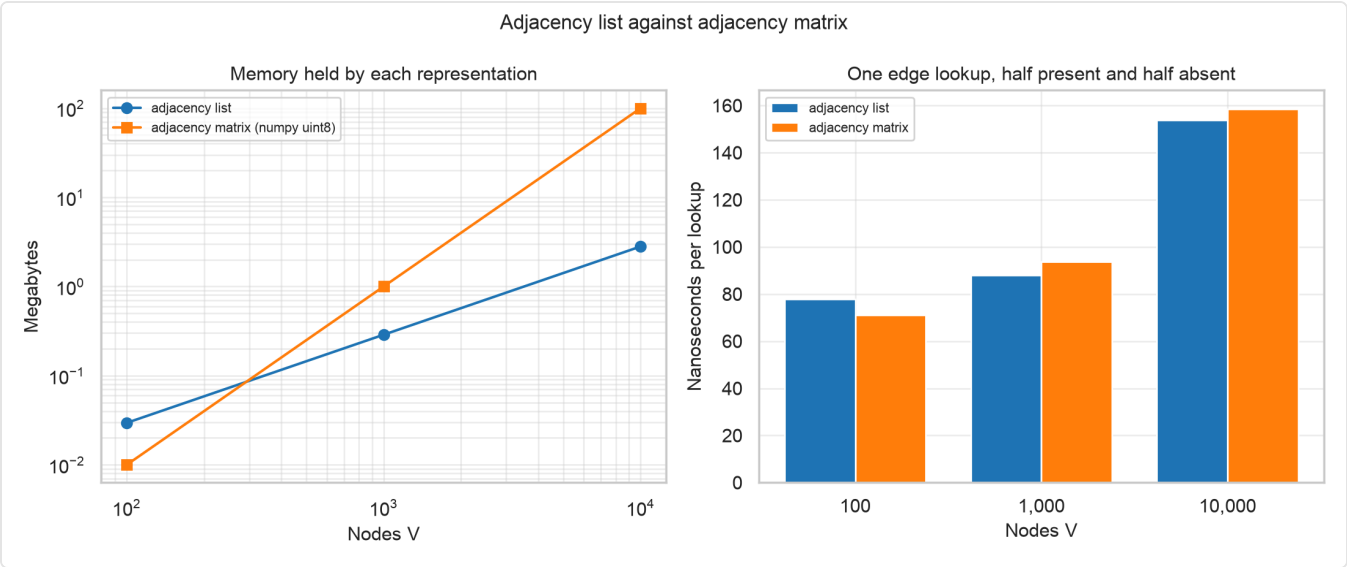
Data. Graphs come from `graph_generator.py`, seeded at 42, so a re-run reproduces every number. Sparse graphs have average degree 4 (E is about $2V$), dense graphs density 0.5, and Dijkstra runs on connected weighted graphs with weights from $[1, 10]$. Correctness is checked off the clock: traversals reach every node once, the DFS forms agree, and both Dijkstra implementations return identical distances.

Sizes and repetitions. Representation at $V = 100, 1,000$ and $10,000$; sparse traversal at 100 through $10,000$; dense traversal at 50 through 800; Dijkstra at 100 through $8,000$. Five measured runs at the smaller sizes, three in the middle, two at the largest, after one or two discarded warm-ups; every CSV row records its own count.

Three reductions, recorded rather than silent. The matrix is numpy backed (`uint8` unweighted, `float32` weighted), so $V = 10,000$ is 100 MB; anything past a 512 MB cap is skipped with the reason written into the row, and no size needed skipping. The linear scan is capped at $V = 2,000$ because it is $O(V^2)$ while the heap runs to $8,000$, so the two series cover different ranges. Dense traversal stops at $V = 800$, since density 0.5 at $V = 10,000$ is 25 million edges, a different experiment rather than a bigger one. Every figure below is recomputed from the CSVs by `tools/week4_facts.py`.

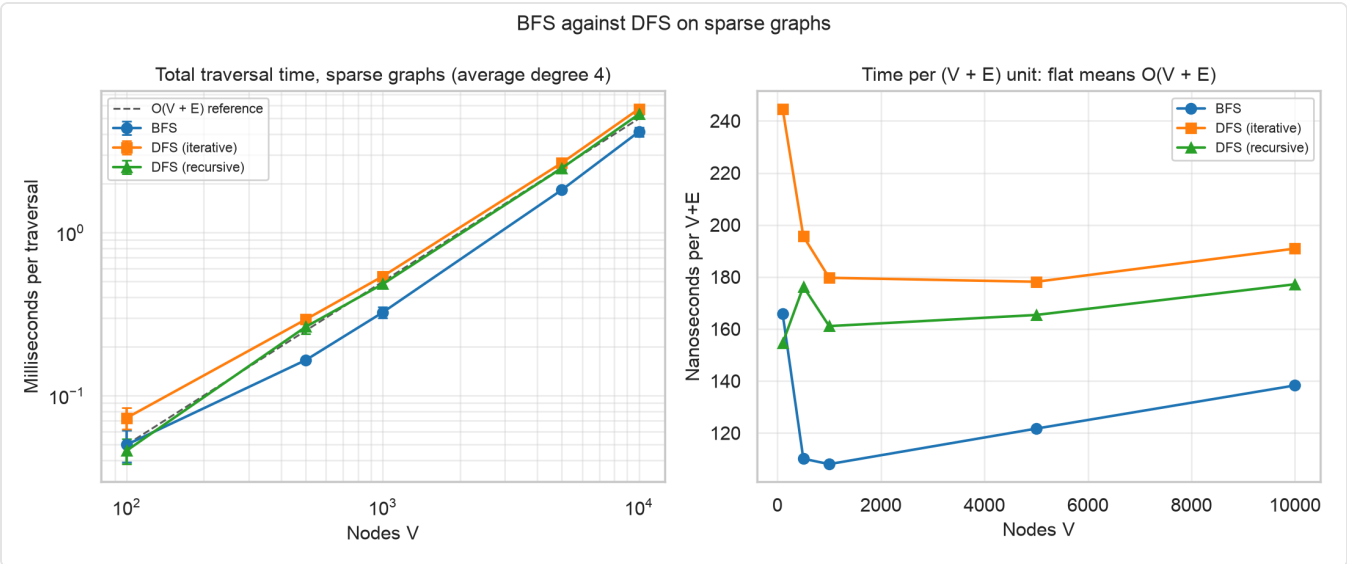
3. Results

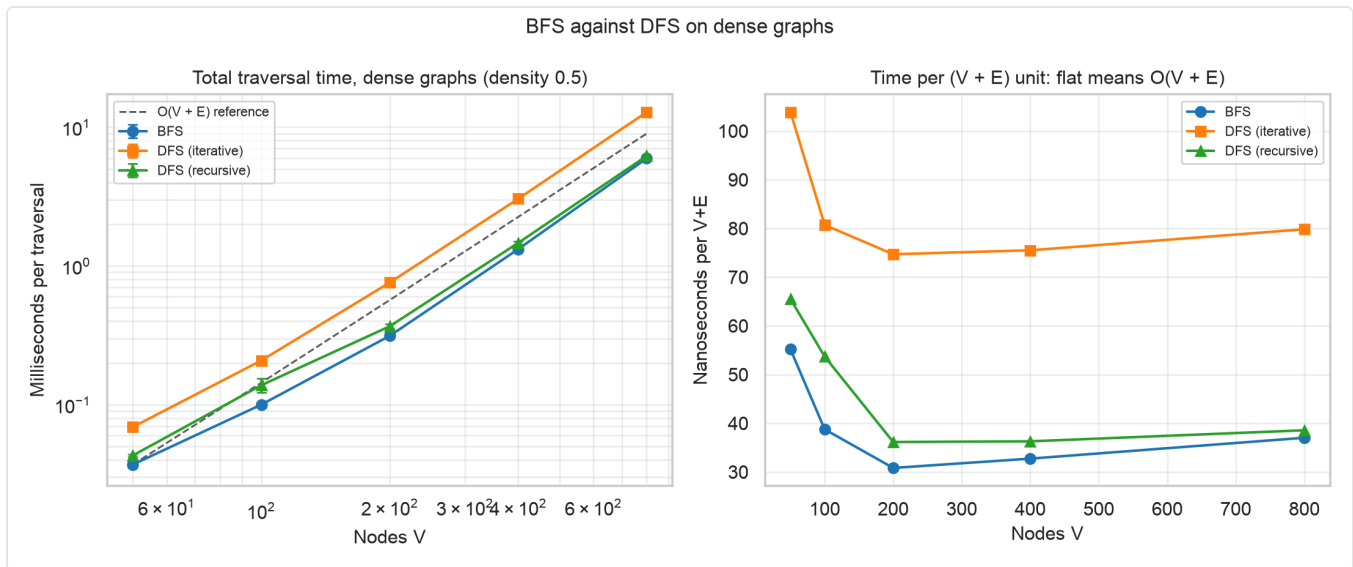
3.1 Representation: list against matrix



Memory did what the bounds require: over a 100x rise in V the list grew 95.1x (slope 0.989), the matrix 10,000.0x (slope 2.000). At V = 100 the matrix is the **smaller** structure, 0.010 MB against 0.030 MB; by V = 1,000 it is 3.5x larger, and by V = 10,000 it is 35.6x larger, 100 MB against 2.808 MB, spending 100,000,000 cells on 20,000 edges at density 0.0004. It buys no speed: one edge lookup, half present and half absent, cost 78.0, 88.1 and 153.8 ns through the list against 71.1, 93.7 and 158.5 ns through the matrix. Both are O(1): a dict hit and a numpy scalar index cost about the same.

3.2 Traversal: BFS against DFS

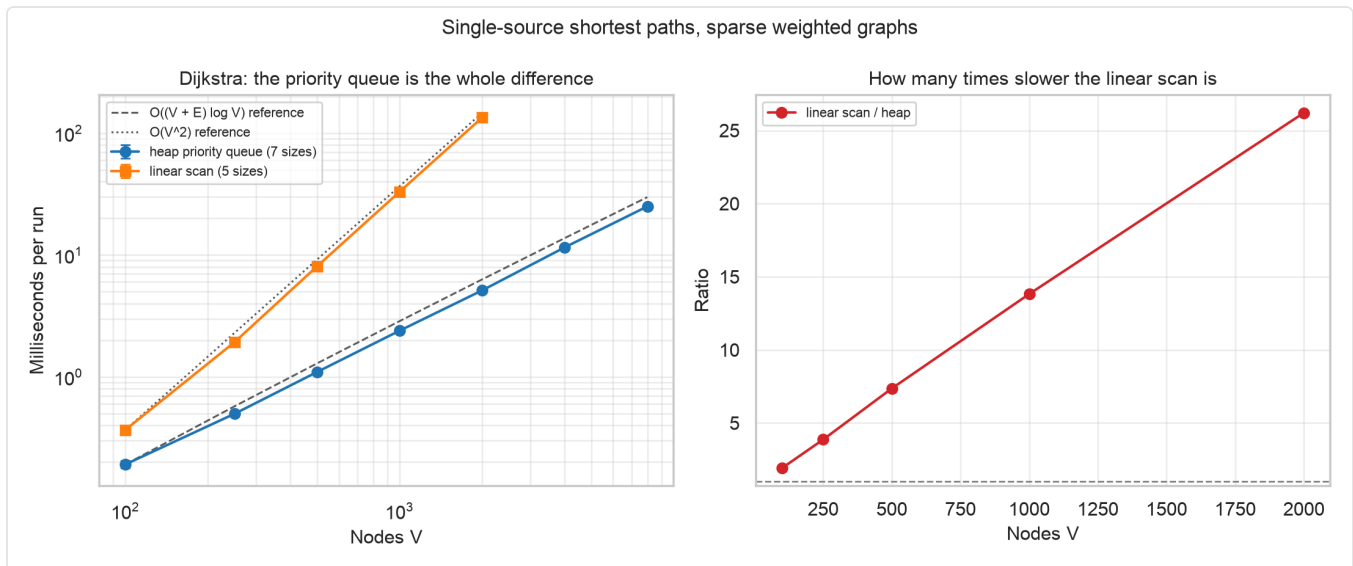




$O(V + E)$ claims that cost per $(V + E)$ unit stays flat, so that is the panel to read. Across a 100x size range BFS held 108.1 to 165.9 ns per unit on sparse graphs, recursive DFS 154.6 to 177.2 ns. Total-time slopes were 0.970, 0.947 and 1.021 against the 1.0 a sparse graph predicts; on dense graphs the same three gave 1.838, 1.895 and 1.775 against 2.0, since E is $O(V^2)$ there.

Two things it does not predict. Per-unit cost is 3 to 4 times **cheaper** on dense graphs (30.8 to 55.2 ns for BFS), so V and E are not interchangeable: a node costs more than an edge. And iterative DFS is slowest everywhere, 2.16x BFS at $V = 800$ on dense graphs while recursive DFS stays at 1.04x.

3.3 Dijkstra: heap against linear scan



Same algorithm, same answers, one structure swapped. The gap widened as the bounds require: 1.92x at $V = 100$, 3.87x at 250, 7.37x at 500, 13.82x at 1,000 and 26.21x at $V = 2,000$, where the scan takes 134.222 ms against the heap's 5.121 ms. The scan's slope is 1.982 against the 2.0 of $O(V^2)$, growing 365.7x against 400.0x predicted. The heap's slope is 1.118, growing 131.0x against 156.1x predicted. That undershoot is worth naming: $(V + E) \log V$ assumes a flat cost per heap operation, which a real heap need not honour.

3.4 Comparison table

Series	Bound	Slope	Growth measured / predicted	Empirical
BFS, sparse	$O(V + E)$	0.970	83.0 / 100.0	$O(n)$
BFS, dense	$O(V + E)$	1.838	160.7 / 256.0	$O(n^2)$

| Dijkstra, heap | $O((V + E) \log V)$ | 1.118 | 131.0 / 156.1 | $O(n \log n)$ | | Dijkstra, linear scan | $O(V^2)$ | 1.982 | 365.7 / 400.0 | $O(n^2)$ |

| Adjacency matrix, memory | $O(V^2)$ | 2.000 | 10,000 / 10,000 | $O(n^2)$ | | Adjacency matrix, lookup | $O(1)$ | 0.174 | 2.23 / 1.00 | $O(\log n)$ * |

Sixteen of the eighteen series in [comparison_table.csv](#) match their expected class. Both disagreements (*) are lookup rows, and the matrix one settles what they mean: `matrix[i, j]` is one array index whose operation count cannot vary with V , and it still grew 2.23x. What grew was the distance to the data, from a 10 KB array in L1 to a 100 MB one in no cache: the classifier is reading the memory hierarchy, not the algorithm.

4. Discussion

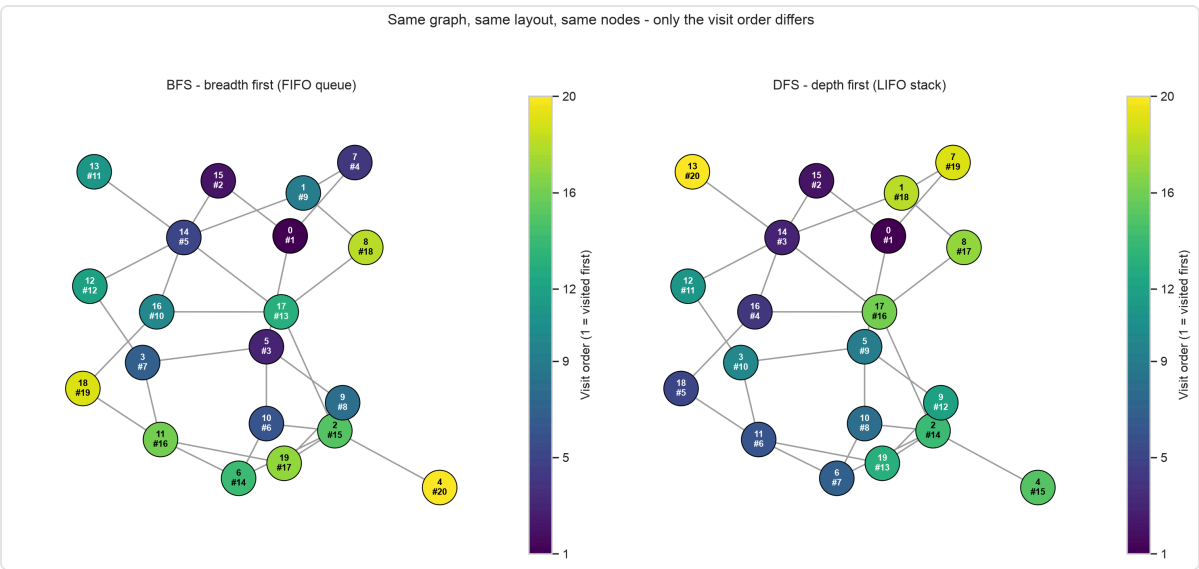
The traversal is one loop, and the container is the algorithm. Take a node from a pending collection, mark it seen, push its unseen neighbours. Make the collection a FIFO `deque` and it is BFS; a LIFO list and the same lines are DFS; a priority queue keyed on distance and it is Dijkstra. Section 3.3 measures it: the queue changed, nothing else did, and the cost moved 26x.

It breaks in two places, both measurable. Recursive DFS is not that loop: it has no explicit container, it borrows the interpreter's call stack, so the container is owned by the runtime rather than the program. The explicit stack holds every unvisited neighbour as it is discovered, growing to $O(E)$ entries and paying a membership check on each, while the call stack holds at most $O(V)$ frames. On dense graphs, where E is 400 times V , that is the whole gap between 2.16x BFS and 1.04x.

Dijkstra breaks it the other way: it needs what the skeleton lacks, edge relaxation, and the ability to revise a node's priority after pushing it. The Week 3 `PriorityQueue` has no decrease-key, so [dijkstra.py](#) uses lazy deletion: push a duplicate at the better distance, discard settled entries on pop. The heap then holds $O(E)$ entries rather than $O(V)$, which is the honest limit of the thesis. The container is necessary; here it stopped being sufficient.

Density chooses the representation, and none of the three algorithms can see it. They all just ask for neighbours. At density 0.0004 the matrix wastes 99.96% of its cells; at 0.5 it is the compact one. Density silently decides whether the structure underneath was well chosen.

5. Visualization Summary



Both panels show the same 20-node graph in the same networkx layout, each node coloured and labelled by its visit step. Only the container differs. BFS colours in rings: one hop from node 0 is dark, the two-hop ring lighter, the last node visited out at the edge. DFS colours in a ribbon that wanders to the boundary and back, so adjacent numbers trace a path, and node 13 moves from step 11 under BFS to step 20 under DFS.

6. Conclusion

Every result here is a trade rather than a winner. The matrix buys constant-time addressing for an $O(V^2)$ bill worth paying only above a density these graphs never reach: 35.6x the memory at $V = 10,000$ for a lookup measuring 1.03x. The heap buys a log factor and pays a duplicate entry per relaxation. Recursive DFS buys a smaller container and pays with a recursion limit raised deliberately to survive a 10,000-node path. Choosing the container is choosing which bill arrives, and the algorithm on top usually cannot tell you which.

References

- Cormen, Leiserson, Rivest, Stein. *Introduction to Algorithms*, 4th ed., Ch. 22, Elementary Graph Algorithms, pp. 589-619 (representations, BFS, DFS), and Ch. 24.3, pp. 658-666 (Dijkstra)
- Dijkstra, E. W. (1959). "A Note on Two Problems in Connexion with Graphs." *Numerische Mathematik* 1(1), 269-271
- Amakobe. *Advanced Computational Algorithms*, 2nd ed.

AI use

Claude (Claude Code) drafted the code, tests and this report's first draft from a specification I wrote from the instructions and required reading. Every measurement comes from running the benchmark on my machine, and every figure is recomputed from the CSVs rather than transcribed. Full disclosure: [docs/AI_USE.md](#) .